CHECK POINT

AI SECURITY · MODEL CONTEXT PROTOCOL · MICROSOFT FOUNDRY

Check Point MCP on Microsoft Foundry — A Best-Practices Guide

The Model Context Protocol (MCP) is how AI agents connect to real tools. This guide explains what MCP is, how a Microsoft Foundry Hosted Agent drives the Check Point `@chkp` MCP servers, and where Check Point fits in its two roles — using this open project as the worked example.

01

Executive Summary

As enterprises connect AI agents to real tools using the Model Context Protocol (MCP), the layer that **aggregates tools, centralizes identity, and enforces policy** becomes the single most important control point for security and governance. This guide makes vendor-neutral MCP best practices the backbone and treats **Microsoft Foundry — a Hosted Agent driving the Check Point `@chkp` servers** — as one concrete worked example, then positions Check Point in its two distinct roles.

2
standard MCP transports: stdio & Streamable HTTP

15
`@chkp` server packages this project hosts (of 18 upstream)

9
servers `deploy` brings up by default

2
model providers behind one loop (Claude on Foundry + `gpt-5-mini`)

Two things to get right, and this guide is built around them:

OFFICIAL — THE GUARDRAILS YOU CAN RUN TODAY

This project ships **two** guardrail engines. The Check Point-native one is the **Check Point AI Guardrail (Lakera Guard)** — one inline `POST api.lakera.ai/v2/guard` that screens the prompt *before* the model, selected with `--guardrail-provider lakera`. It is Check Point's **own GA AI security** (post-Lakera-acquisition) and is *identical on AWS and Azure*. The platform default is **Azure AI Content Safety *Prompt Shields*** (Azure's screening, not a Check Point product). Note: the deeper Check Point **agent/MCP runtime protection** (beyond prompt screening) is still **Early Access** — contact Check Point. Don't present Prompt Shields itself as "Check Point protection."

REFERENCE ARCHITECTURE — LABEL IT AS SUCH

"**Check Point `@chkp` MCP servers on Microsoft Foundry**" — the hosted agent, the stdio children, and the opt-in remote tier — is a pattern this project **assembles from GA Azure + Check Point building blocks**. It was **verified end-to-end on `gpt-5-mini`**, but there is **no announced joint Microsoft + Check Point solution**. Present it as a synthesized reference design, never as a shipped joint product.

02

MCP Fundamentals Worth Understanding

MCP is an open, JSON-RPC 2.0-based protocol that standardizes how AI applications connect to external context and tools — think **"USB-C for AI tools."** It is vendor-neutral: the same five concepts hold on Foundry, on AWS, or on a laptop. Five to be fluent in:

- **Client–host–server architecture.** The *host* (the AI application) creates and manages *client* instances, controls permissions and consent, and coordinates the LLM. Each *client* holds an isolated, stateful session with exactly one *server* (1:1). The 1:1 is directional — one remote server can serve many clients.
- **Stateful, capability-negotiated sessions.** Client and server declare supported features in an initialization handshake. (Evolving: a subset can run stateless over Streamable HTTP, and the upcoming `2026-07-28` revision introduces a stateless protocol core.)
- **Three server primitives:** *tools* (model-controlled actions, discovered via `tools/list` and invoked via `tools/call`), *resources* (application-controlled data via URIs), and *prompts* (user-controlled templates). Clients may offer sampling, roots, and elicitation.
- **Two standard transports:** `stdio` (local subprocess over stdin/stdout) and `Streamable HTTP` (remote; HTTP POST/GET with optional SSE, session via the `Mcp-Session-Id` header). Streamable HTTP replaced the deprecated 2024-11-05 HTTP+SSE transport. This project uses **both**: stdio for the default in-agent path, Streamable HTTP for the opt-in remote tier.
- **Authorization is OAuth 2.1-based, OPTIONAL, and HTTP-only.** stdio servers read credentials from the environment. On HTTP, the MCP server is an OAuth 2.1 resource server; the spec mandates PKCE, RFC 9728 Protected Resource Metadata for discovery, and RFC 8707 resource indicators so tokens are **audience-bound** to a specific server — on Azure, Entra Easy Auth's `allowedAudiences` is exactly this audience check.

> **VERIFIED HANDS-ON**
>
> Driving a live handshake against the real Check Point `@chkp` servers negotiated protocol `2025-06-18` cleanly: `initialize` → `notifications/initialized` → `tools/list` (the same `scripts/mcp_probe.mjs` path this repo ships). The isolation model — a server cannot read the full conversation or see into other servers — is exactly what makes a single *control point* the natural place to concentrate identity and policy.

03

The MCP Control-Point Pattern (The Backbone)

An **MCP control point** (an aggregating agent, or a dedicated gateway) sits in front of one or more MCP servers to provide a single, secure entry point for AI clients. Its jobs:

- **Aggregate** many servers behind one place (a unified tool catalog).
- **Centralize authentication / authorization** across all backends.
- **Enforce policy & tool allowlists** — what may be called, by whom.
- **Broker credentials server-side** so agents never see secrets.
- **Provide observability / audit** — one place for logs and metrics.
- **Route & version** — session affinity, backend lifecycle.

Why control points exist — the configuration explosion

Docker frames it precisely: a control point turns an *X applications* × *Y servers* problem into just *Y*. Each client connects once, and the control point manages every backend behind a unified interface — collapsing N-clients × M-servers sprawl into one connection per client. On Foundry, the **Hosted Agent itself is the control point** for its own use (it spawns the `@chkp` servers as stdio children); when a *second* consumer needs the same tools, the opt-in remote tier promotes each server to a shared, authenticated endpoint.

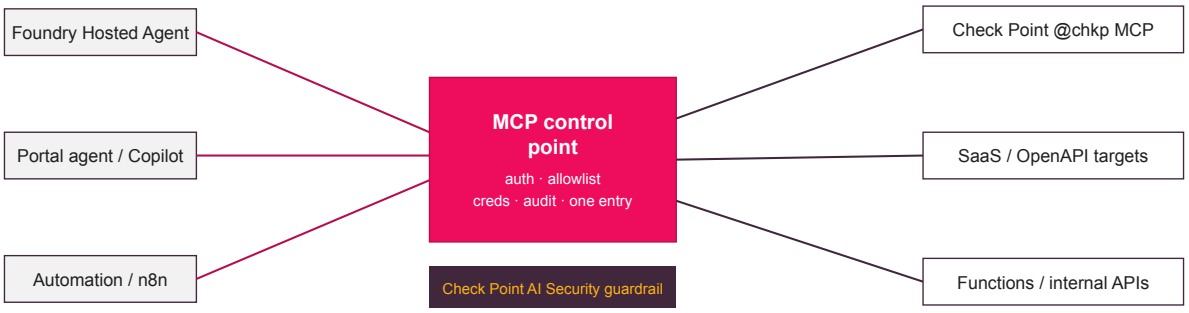

Figure 1 — The control point is the choke point: clients connect once; it fronts many MCP servers, brokers credentials, and is where a runtime guardrail (e.g. Check Point AI Security, or Azure Content Safety) can enforce allow/deny outside the agent's reasoning loop.

Pattern categories & ecosystem

A commonly-cited (community, *not* spec-defined) taxonomy has five categories: transport bridging, server aggregation, security gateway, cloud-native gateway, and edge-based patterns. Present it as convention, not standard.

Category	Representative implementations
Open-source	mcp-proxy, supergateway, IBM ContextForge (mcp-context-forge), MetaMCP, Docker MCP Gateway, Lasso, Envoy AI Gateway
Cloud-managed	**Microsoft Foundry project Toolbox**, Azure API Management (AI gateway), AWS Bedrock AgentCore Gateway, Kong, WSO2

TRADE-OFF — SAY THIS OUT LOUD

A control point **concentrates trust and risk into one control plane**. That is a high-value attack surface and a potential single point of failure — it must itself be hardened, monitored, and made highly available. On Foundry that plane is the Hosted Agent (and, when enabled, the remote Container-App tier behind Entra Easy Auth).

04

MCP Best-Practices Checklist

The vendor-neutral controls a control point (or a well-run MCP deployment) should implement. Items marked **SPEC** are normative MUST / MUST-NOT requirements of the MCP authorization and security specs; the rest are industry consensus. The *Azure mapping* notes show how this project satisfies each on Foundry.

Identity & tokens

- **SPEC.** Enforce OAuth 2.1 + PKCE (S256); use RFC 9728 for AS discovery and RFC 8707 resource indicators so tokens are **audience-bound**. Validate audience server-side even if the authorization server ignores the resource parameter — that check is the real enforcement point. *Azure mapping:* **Entra Easy Auth** `allowedAudiences` (`api://<clientId>`, `Return401` for anything without a valid token) is the audience check on the remote tier.
- **SPEC. Never pass client tokens through to upstream APIs.** Token passthrough breaks audit, bypasses rate limits, and opens a confused-deputy / exfiltration path. Obtain a separate downstream token. *Azure mapping:* inbound is Entra (Easy Auth / Foundry-authenticated Responses); the server-to-Check-Point credential is a **separate** secret held in Key Vault.
- Issue short-lived, scoped tokens; rotate refresh tokens for public clients. Practice **scope minimization** — start minimal and elevate progressively; avoid wildcard/omnibus scopes. *Azure mapping:* least-privilege Entra RBAC — the agent identity gets exactly `Cognitive Services User` + `Key Vault Secrets User`; the remote identity gets exactly `AcrPull` + `Key Vault Secrets User`.
- Never use MCP sessions as an authentication mechanism; use non-deterministic session IDs bound to user identity.

Tools & trust

- **Pin and hash tool definitions** (name + description + schema); alert / re-prompt on any change to defeat "rug-pulls." Version-lock MCP server packages. *Azure mapping:* every `@chkp` server is pinned (`npx -y @chkp/<server>-mcp@<pin>`) and the agent image is digest-pinned.
- Maintain a **tool/server allowlist**; inspect full descriptions, parameter names, and return schemas before approval (scan for hidden or AI-only instructions). Use strict JSON Schema with `additionalProperties:false`.
- Require **human-in-the-loop** for destructive / financial / data-sharing / cross-server calls, showing full parameters — never summaries. Never auto-approve multi-server calls.
- Treat every tool *result* as untrusted input; scan/sanitize before returning it to the model. Treat third-party tool annotations as untrusted.

Network & operations

- Broker credentials server-side and inject at execution time; **decouple inbound (client→control point) from outbound (control point→backend) auth.** *Azure mapping:* **Key Vault via managed identity** — the secret is fetched at container/child start, never baked into the image, git, or the container's env map.
- Apply egress / SSRF defenses: block private/reserved ranges and cloud metadata (`169.254.169.254`), require HTTPS, guard against DNS-rebinding, route server-side fetches through an egress proxy.
- Validate `Origin` headers; **bind local servers to** `127.0.0.1` (not `0.0.0.0`); require auth on all HTTP connections. *Azure mapping:* the remote tier is external only behind Easy Auth with ACA-managed TLS (`allowInsecure: false`) — never a raw port.
- Centralize observability: log every invocation with parameters, user, and timestamp to a SIEM (redacting secrets/PII). Isolate sensitive servers on separate networks; run backends least-privileged. *Azure mapping:* OpenTelemetry → Application Insights / Log Analytics; container stdout via the session log stream.
- Vet the supply chain: install only from verified sources, review source and tool definitions, verify checksums/signatures, watch for typosquatting.

The MCP Threat Model — What To Defend Against

MCP inherits classic AppSec / OAuth risks *plus* a new agent-specific class where the LLM treats tool metadata and results as trusted instructions. Distinguish spec-codified threats from research taxonomy.

SPEC-CODIFIED (MCP SECURITY BEST PRACTICES + AUTHORIZATION)

Threat	Essence & mitigation
Token passthrough	MUST NOT accept/forward tokens not issued for the server; MUST validate audience.
Confused deputy	A proxy using a static client_id with a third-party AS can leak auth codes via an existing consent cookie. Mitigate: per-user approved-client registry checked before forwarding; exact redirect_uri matching; single-use state; hardened consent cookies.
SSRF	Malicious internal URLs in OAuth discovery fields. Block private ranges + metadata, HTTPS-only, DNS-rebinding guards, egress proxy.
Session hijacking	Verify every inbound request; don't use sessions for auth; non-deterministic IDs bound to user identity.
Local server compromise	Show the exact untruncated install command, require approval, sandbox, flag dangerous patterns (`sudo`, `rm -rf`).

RESEARCH / COMMUNITY TAXONOMY (DO NOT PRESENT AS SPEC TERMS)

- **Tool poisoning** — malicious instructions hidden in tool descriptions/schemas, invisible to users but read by the model. Invariant Labs' April 2025 PoC exfiltrated `~/.ssh/id_rsa` via a benign-looking "add" tool. The highest-severity MCP-specific class.
- **Rug-pull / mutable definitions** — a server silently changes a tool after approval, because most clients bind trust to the tool *name*, not its content. One-time vetting is insufficient.
- **Tool shadowing / name collision** — a malicious server's description manipulates the agent's use of a *different*, trusted server.

> **PROVEN IN THE WILD**
>
> The **postmark-mcp** npm package (v1.0.16, Sept 2025) silently BCC'd every outgoing email to an attacker domain after ~15 clean, reputation-building releases — the first confirmed real-world malicious MCP server (~1,500 weekly downloads). **Lesson: change-detection and re-verification over time, not just install-time vetting.**

Worked Example: Microsoft Foundry

Microsoft Foundry's **Hosted Agent** is a managed, bring-your-own-container runtime: you ship an image to Azure Container Registry (ACR), Foundry runs it as a per-session sandbox behind an Entra-authenticated HTTPS endpoint speaking the **Responses** `2.0.0` protocol, auto-injects Application Insights, and treats each published version as **immutable** (a `refresh` bumps the version to restart sandboxes). This project builds its agent image remotely with `az acr build` — no local Docker required — and creates the hosted agent (`chkpmcp-agent`, port 8088) via `azure-ai-projects`.

The default topology — stdio children, no gateway

By default the `@chkp` servers run as **stdio child processes the agent spawns itself** (`npx -y @chkp/<server>-mcp@<pin>`) — the *same* code path in the local runtime (`chat --runtime local`) and inside the hosted container. There is **no gateway tier by default**: zero extra infrastructure, lowest latency, and the credentials never leave the process. The trade-off is scope — **only this agent can use the tools**. (On AWS the identical servers sit behind one AgentCore Gateway; that aggregation tier is AWS-only here.) Tools are merged and namespaced `«server»___«tool»` (e.g. `quantummanagement___show_hosts`) in `mcp_stdio.py`.

The opt-in remote MCP tier — a second consumer

`deploy --remote-mcp` stands up a remote tier **alongside** (not replacing) the stdio path — the Azure analogue of the AWS AgentCore Gateway. Each selected `@chkp` server *also* runs as its own **scale-to-zero Azure Container App** in the packages' native streamable-HTTP transport (reusing the same agent image with command `python -m chkpmcpaz.remote_server`), fronted by **Entra Easy Auth** (`Return401` for anything without a valid token for the gateway app-registration audience). A **second consumer** — Foundry portal agents, Copilot Studio, Claude Desktop, or any other MCP client — can then share the tools. Credentials still come from **Key Vault via a managed identity** (AcrPull + Key Vault Secrets User); **no secrets in the containers**. Scale-to-zero keeps idle cost ~0.

The model story — one loop, two providers

The agent is **provider-agnostic behind one tool loop**. The production path is **Claude on Foundry** — Anthropic Messages via the account's `/anthropic` route with the `AnthropicFoundry` client and Entra bearer tokens (`claude-sonnet-4-6` primary, `claude-haiku-4-5` fallback). For cheap testing, the *identical* loop runs on a first-party **`gpt-5-mini`** Azure OpenAI deployment (OpenAI Chat Completions) — the Azure analogue of falling back to Amazon Nova on Bedrock. A small provider seam (`get_provider` → `AnthropicProvider` | `AzureOpenAIProvider`) speaks each vendor's native wire format; the provider is auto-detected from the model name or forced with `--provider` / `CHKP_PROVIDER`.

Auth & secrets — Entra ID throughout

- **Entra ID everywhere:** `DefaultAzureCredential` locally, a per-agent Entra identity in the hosted sandbox, and Entra Easy Auth on the remote tier. No API keys, no custom auth tier — **every endpoint requires authentication**.
- **Secrets in Key Vault** (one secret per credentialed server, `<prefix>-<server>`); placeholder bodies at deploy, real values loaded by `creds apply` and never clobbered, never printed or logged.
- **TLS 1.2+** for data in transit; the ACR registry is Entra-only (admin user disabled); the remote tier uses ACA-managed TLS.

OBSERVABILITY — DO NOT OVER-CLAIM

The hosted agent emits **OpenTelemetry** that flows to **Application Insights / Log Analytics** (`appi-` / `log-<prefix>-<token>`); the connection string is auto-injected into sandboxes. Container **stdout is retrievable via the session log stream**. That is the honest scope — traces, logs, and KQL over them. Do not promise a specific dashboard, metric name, or third-party export unless you have confirmed it against current Azure docs for your region.

07

Where Check Point Fits — Two Complementary Roles

A customer must not conflate these. Check Point both *publishes MCP servers* (tools an agent can call) and *sells the AI-security enforcement layer* that polices agentic/MCP traffic.

Role 1 — Check Point as MCP tools

The official open-source monorepo github.com/CheckPointSW/mcp-servers ships MIT-licensed `@chkp/` packages; this project hosts **15 of the 18 upstream** (Quantum Management, Management Logs, Threat Prevention, HTTPS Inspection, Policy Insights, GW-CLI, Reputation, Threat Emulation, Documentation, CloudGuard WAF, Spark, Argos ERM, Harmony SASE, Workforce AI, Quantum Gaia). They let an agent query your estate in natural language.

- **Transport:** default `stdio` via `npx -y @chkp/<pkg>@<pin>`, spawned as children of the agent. An optional HTTP mode exists but historically shipped with no built-in auth and no TLS (see gotcha below); this project only exposes it through the Easy-Auth-fronted remote tier.
- **Auth (env vars, never shown to the model):** Smart-1 Cloud (`API_KEY` + `S1C_URL`); on-prem (`MANAGEMENT_HOST` / `PORT` + `API_KEY` or `USERNAME` / `PASSWORD`); ThreatCloud / Infinity Portal keys for the others. Values come from Key Vault at spawn.
- **Read-oriented — but verify per package.** Quantum Management & Harmony SASE self-describe as read-only; several packages say "manage." Scope the API key to least privilege regardless.

VERIFIED HANDS-ON

`@chkp/quantum-management-mcp` (v1.4.7) exposed a read-only catalog (`show_*`, `where_used`, `simulate_packet`, `find_zero_hits_rules`, plus a `management__init` login tool). Tool *registration* is static, so `tools/list` works even before the management API is reachable — useful for a smoke test with placeholder credentials.

Role 2 — Check Point as the enforcement layer

Check Point's "AI Defense Plane" spans Workspace AI security, AI application security, and **AI agent runtime security** (governing tool calls, blocking unsafe autonomy). Verified specifics:

- **CloudGuard WAF + Lakera dual-layer ML** — a supervised layer (Prompt Injection Prevention, Data Leakage Prevention, Content Control, Usage Control) plus an unsupervised refinement layer, 100+ languages, covering GenAI apps, APIs & agents.
- **Quantum Firewall R82.10 natively detects & monitors MCP traffic** — a shipped capability (Dec 2025 release), including auto-detection of unauthorized GenAI tools and MCP servers.
- **Lakera:** Check Point announced an agreement to acquire Lakera (Sept 2025). *Gandalf* — Lakera's adversarial red-team game — *feeds* the guardrails; the product is **Lakera Guard / AI Guardrails**. Do not equate Gandalf with the guardrail engine.

THE CENTRAL TECHNICAL GOTCHA

Check Point MCP servers are `stdio` / `npx` by default, and their optional HTTP mode (`--transport http` , port 3000, `/mcp` + `/health`) historically shipped with **no authentication and no TLS** — the README says to run it only behind an authenticated, TLS-terminating reverse proxy. This project fixes that for the remote tier by putting **Entra Easy Auth + ACA-managed TLS** in front of every endpoint (`allowInsecure: false` , `Return401`). There is no hosted/managed remote Check Point endpoint from the vendor; servers run in the customer's environment. **Never expose a raw `@chkp` MCP HTTP port.**

TLS POLICY FINDING — ON-PREM CLIENT

Separately, the on-prem management client historically hardcodes `new https.Agent({ rejectUnauthorized: false })` in its `login()` / `callApi()` path — it **disables TLS certificate verification** on the connection to `MANAGEMENT_HOST` . A self-signed SMS cert therefore won't block it, but this **conflicts with the "never disable TLS verification" rule** (and `NODE_EXTRA_CA_CERTS` is inert while it's forced). **Smart-1 Cloud** (`S1C_URL`) keeps verification *on* — prefer it, or fork/patch the client to trust the SMS CA. Worth flagging to the `@chkp` product team.

08

Check Point + Microsoft Foundry: Official vs. Reference Architecture

This is the section to get exactly right. There are two separate stories, and neither is a shipped joint Microsoft + Check Point product.

OFFICIAL — THE GUARDRAILS YOU CAN RUN TODAY

Two engines screen the prompt *before* the model (`chat --guardrail`, or baked into the hosted agent with `deploy --guardrail` / `CHKP_GUARDRAIL=enforce`). The **Check Point-native** one is the **Check Point AI Guardrail (Lakera Guard)** — `--guardrail-provider lakera`, one inline `POST api.lakera.ai/v2/guard`, GA and identical on AWS and Azure. The platform default is **Azure AI Content Safety *Prompt Shields*** (`POST {endpoint}/contentsafety/text:shieldPrompt`) — **Azure's** screening, not a Check Point product. Deeper Check Point **agent/MCP runtime protection** (beyond prompt screening) remains **Early Access** — contact Check Point.

SYNTHESIZED REFERENCE ARCHITECTURE — NO OFFICIAL JOINT BLUEPRINT

The "`@chkp` servers on Foundry (stdio + opt-in remote tier)" architecture is assembled from GA Azure + Check Point building blocks and was **verified end-to-end on `gpt-5-mini`**. Every step is individually grounded in Azure and Check Point docs — but it is **not an announced joint solution**. Label it clearly when presenting.

How the reference architecture works

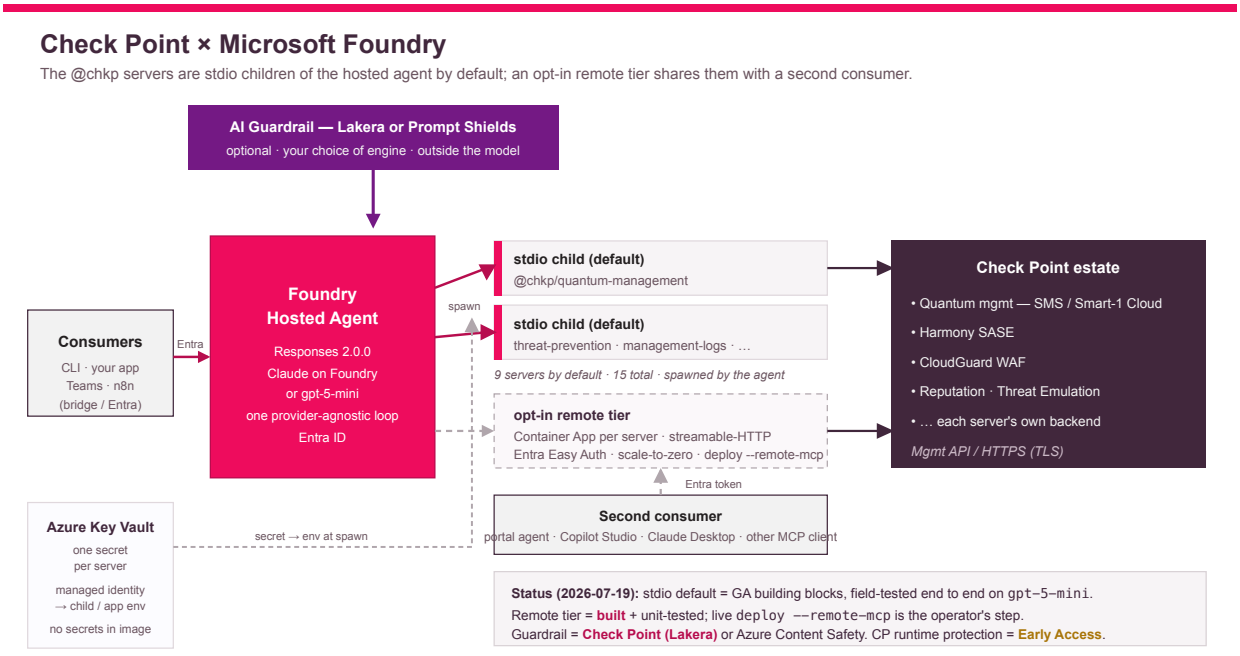

Figure 2 — The Foundry reference architecture. Consumers reach a single Hosted Agent (Claude or `gpt-5-mini`) that spawns the `@chkp` servers as stdio children by default; the dashed opt-in remote tier promotes each server to an Entra-Easy-Auth Container App for a second consumer. Key Vault holds per-server credentials (managed identity, no secrets in the image); the optional inline guardrail is the Check Point AI Guardrail (Lakera) or Azure Content Safety Prompt Shields.

1. **Provision the infra.** `deploy` runs `azd provision` over Bicep: the Foundry account + project, the model deployments (Claude, or first-party `gpt-5-mini`), Key Vault, ACR, and Log Analytics / Application Insights, with least-

privilege RBAC declared before the model LRO completes so it propagates during the wait.

2. **Build the agent image.** `az acr build --file agent/Dockerfile .` builds remotely in ACR (Python 3.12 + Node 20, linux/amd64, port 8088) — no local Docker. The registry is Entra-only (admin user disabled) and the image is digest-pinned.
3. **Create the Hosted Agent.** The CLI creates `chkpmcp-agent` via `azure-ai-projects` (Responses protocol `2.0.0`), then grants its per-agent Entra identity `Cognitive Services User` (Claude route + Content Safety) and `Key Vault Secrets User` — the identity only exists after the version is created.
4. **Seed and load credentials.** Placeholder secrets at deploy; real values via `creds apply` (or `deploy --creds`) straight into Key Vault — never printed, never in git. The hosted sandboxes re-read them after a `refresh`.
5. **Optional — stand up the remote tier.** `deploy --remote-mcp` registers the Entra app (the Easy Auth audience `api://<clientId>`), a shared user-assigned identity (AcrPull + Key Vault Secrets User), a Container Apps environment, and one scale-to-zero Container App per selected server behind Easy Auth. The endpoint catalog is persisted for a second consumer.
6. **Ask the agent.** `chat` runs the reason → `@chkp` tools → loop, grounded in real tool output, with prompt caching, streaming, and per-run token telemetry — locally or on the hosted runtime with platform-managed conversation history (`--session`).

> **FIELD-TESTED BUILD AVAILABLE**
>
> The **local** and **hosted** chat paths were proven **end to end on** `gpt-5-mini` — the full chain (hosted agent → provider → `@chkp` MCP tools → grounded answer). The **remote MCP tier is built and unit-tested (372 tests pass) and its Bicep compiles**; a live `deploy --remote-mcp` against a real subscription is the operator's step. The pure-logic test suite runs on Python 3.11/3.12/3.13. Step-by-step commands are in the repo README and the scenario runbooks (`docs/scenarios/`).

> **THE IDEAL COMBINED TOPOLOGY**
>
> The `@chkp` tools on Foundry *and* Check Point's own runtime AI protection policing that same agentic usage is the natural end state — but Check Point runtime protection is **Early Access**, not a documented, supported Foundry integration. Present it as a roadmap/open question, not a promise; enforce today with Azure Content Safety and Check Point's inline Guard API.

09

The Integration Ways — Choosing Your Mode

Check Point meets Microsoft Foundry across three families — **Tools** (the agent uses Check Point), **Enforcement** (Check Point or Azure polices the agent), and **Network/Edge** (Check Point secures the traffic). Statuses below are honest **as of**

19 July 2026: several Tools ways are GA building blocks field-tested on `gpt-5-mini`; the marquee Check Point runtime guardrail is **Early Access**.

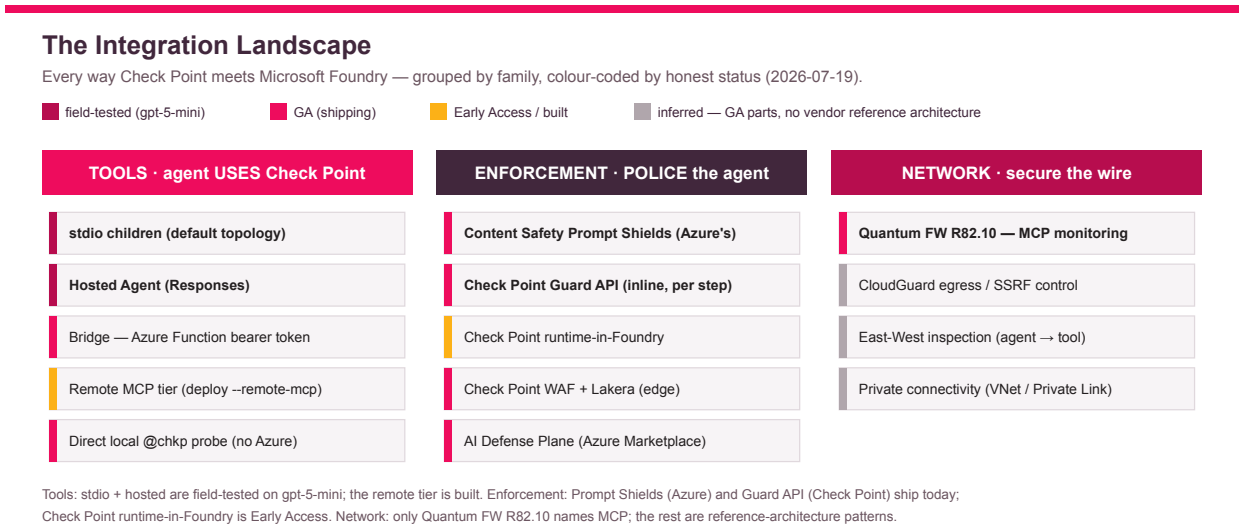

Figure 3 — The integration landscape: three families — Tools (the agent *uses* Check Point), Enforcement (Check Point / Azure *polices* the agent), and Network (Check Point secures the *traffic*) — colour-coded by honest shipping status.

Family	Headline way	Provider	Status
Tools	**stdio children** — `@chkp` servers spawned by the agent (default)	`@chkp` MCP servers	GA parts · **field-tested on** `gpt-5-mini`
Tools	**Hosted Agent** — the same loop on Foundry (Responses)	Foundry Hosted Agent	**live-validated**
Tools	Bridge — Azure Function bearer token for Teams / n8n / curl	Azure Function	**runnable**
Tools	Remote MCP tier — shared Entra-Easy-Auth Container Apps (`deploy --remote-mcp`)	Azure Container Apps	**built** (operator to deploy live)
Tools	Direct local `@chkp` probe (no Azure)	`@chkp` MCP servers	**GA** (Check Point MCP behavior)
Enforcement	Content Safety **Prompt Shields** — screen before the model	Azure (not Check Point)	**GA**
Enforcement	Check Point **Guard API** — inline enforcement per agent step	Check Point AI Agent Security	**GA**
Enforcement	Check Point **runtime-in-Foundry** guardrail	Check Point	**Early Access** (contact Check Point)
Network	Quantum Firewall R82.10 — network-level MCP monitoring	Quantum Firewall R82.10	**GA**
Network	CloudGuard egress / SSRF control + private connectivity	CloudGuard Network Security	inferred (parts GA)

Which do I want? To let the agent *use* Check Point → **Tools** (start with stdio children — the default — or the direct local probe for a no-Azure POC; add the remote tier only when a *second* consumer needs the same tools). To *police* the agent → **Enforcement** (enforce today with Azure's Prompt Shields and/or Check Point's inline Guard API; Check Point *runtime-in-Foundry* is Early Access — plan, don't promise). To secure the *wire* → **Network** (Quantum Firewall R82.10 is the only shipping product that names MCP; add CloudGuard egress/SSRF control).

READ CAREFULLY — WHAT IS AND ISN'T SHIPPED

The **remote MCP tier is opt-in and built in *this project*** — it is **not** a Microsoft-shipped Check Point connector. **Prompt Shields is Azure's** screening, not Check Point's. **Check Point runtime-in-Foundry is Early Access.** What is GA and buildable today is the Azure substrate (Foundry Hosted Agents, Container Apps + Easy Auth, Content Safety) plus Check Point's own inline engine (Guard API). Wire Check Point's runtime guardrail in when the official Early-Access mechanism is available to you.

10

Summary & Honesty Guardrails

1. **The problem.** Connecting AI agents to real tools via MCP creates (a) configuration sprawl, (b) a new attack class where tool descriptions and results become injection vectors, and (c) credential/audit gaps because MCP auth is optional and HTTP-only. Real incidents already exist (postmark-mcp, Sept 2025).
2. **The pattern.** A control point solves all three: one entry per client, centralized identity with audience-bound tokens (Entra Easy Auth on the remote tier), tool allowlisting/pinning, human-in-the-loop, credential brokering (Key Vault via managed identity), and unified audit.
3. **Microsoft Foundry** is a mature managed example: a Hosted Agent (BYO container, Responses `2.0.0`, immutable versions, App Insights) that drives the `@chkp` servers as stdio children — with an opt-in remote tier when a second consumer needs the tools.
4. **Check Point's two roles:** *tools* (the `@chkp` MCP servers query your estate in natural language; credentials stay in Key Vault and are never shown to the model) and *enforcement* (CloudGuard WAF + Lakera dual-layer ML and Quantum Firewall R82.10's native MCP monitoring provide runtime guardrails the platform alone doesn't give you).
5. **The Claude subscription caveat — say it up front. Claude on Foundry requires a pay-as-you-go / EA subscription.** MSDN / Visual Studio / Dev-Test / free-trial / CSP subscriptions are **blocked** — the tool's `doctor` preflight fails them before any deploy. When Claude is unavailable, demo the cheap first-party **`gpt-5-mini`** path (the Azure analogue of Amazon Nova): it deploys on Dev-Test subscriptions, needs no `--org` and no Marketplace terms, and exercises the identical tool loop.
6. **The guardrail caveat.** The Azure-native guardrail this project can run today is **Azure's Content Safety Prompt Shields — not Check Point's runtime protection**, which is **Early Access** (contact Check Point). Never present Prompt Shields as a Check Point product.
7. **The remote-tier caveat.** The remote MCP tier is **opt-in and built in this project** — it is **not** a Microsoft-shipped Check Point connector. And the "`@chkp` servers on Foundry" architecture is a synthesized reference design verified on `gpt-5-mini`, not an announced joint Microsoft + Check Point solution.
8. **Recommend defense in depth:** least-privilege Entra RBAC, secrets in Key Vault (never in code/git/env), Entra Easy Auth on every endpoint, digest-pinned images — *plus* Check Point runtime guardrails in front of agentic usage.

VERSION-SENSITIVITY NOTICE

This is a fast-moving area. As of : MCP spec baseline is `2025-11-25` (a `2026-07-28` release candidate exists but is not yet published); ; shipped at Build 2026; the `@chkp` packages are (e.g. `@chkp/quantum-management-mcp@1.4.7`) — verify each pin before quoting.

Node.js 20+ required for the Check Point servers; disable telemetry with `TELEMETRY_DISABLED=true`.

> Prepared for customers who want to experiment with Check Point's open-source MCP servers. The "`@chkp` MCP servers on Microsoft Foundry" design is a synthesized reference architecture, not an announced joint Microsoft + Check Point solution. Verify all claims against current primary documentation.

PRIMARY SOURCES

MCP specification — modelcontextprotocol.io/specification/2025-11-25

Check Point MCP servers — github.com/CheckPointSW/mcp-servers

Microsoft Foundry Hosted Agents — learn.microsoft.com/azure/ai-foundry — hosted agents

Claude in Microsoft Foundry — learn.microsoft.com/azure/ai-foundry — Anthropic / Azure Marketplace

Azure Container Apps authentication (Easy Auth) — learn.microsoft.com/azure/container-apps/authentication

Azure AI Content Safety Prompt Shields — learn.microsoft.com/azure/ai-services/content-safety

OWASP MCP Security Cheat Sheet — cheatsheetseries.owasp.org

postmark-mcp incident — snyk.io